# THE UNIVERSITY OF AZAD JAMMU & KASHMIR, MUZAFFARABAD

| | |
|---|---|
| COURSE TITLE | CALD |
| COURSE CODE | CS-1205 |
| LAB TITLE | EMULATOR |
| STUDENT NAME | HASEEB SHEIKH |
| STUDENT ROLL NUMBER | 2024-SE-31 |
| INSTRUCTOR NAME | ENGR. SIDRA RAFIQUE |
| SUBMISSION DATE | AUG 28, 2025 |

## DEPARTMENT OF SOFTWARE ENGINEERING

# CALD

## Lab 8

### Objective:

To demonstrate basic arithmetic operations in x86 assembly using Emu8086 and observe their impact on CPU registers.

# Emulator 8086

## Addition Operation

### Original Code:

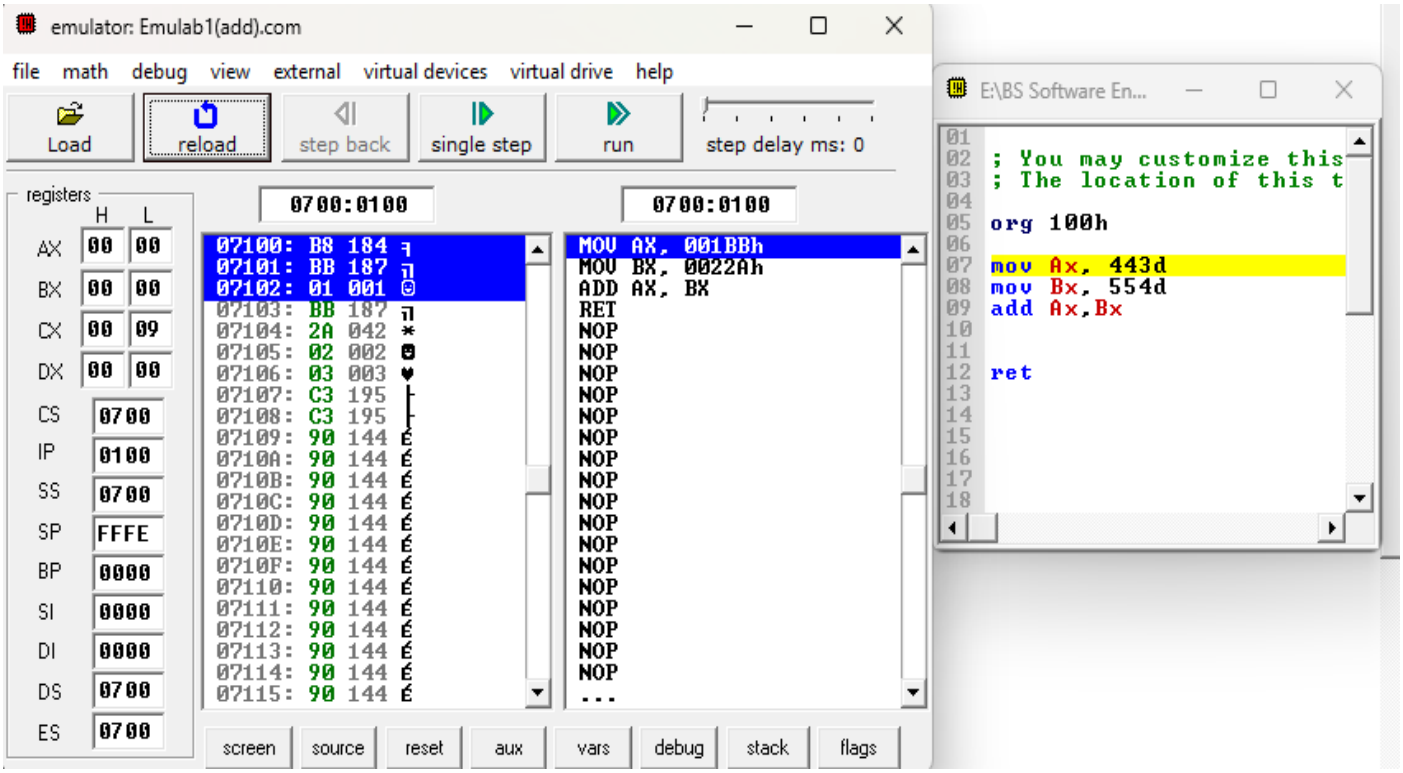
```
org  100h

mov  Ax,  443d
mov  Bx,  554d
add  Ax,Bx


ret
```

### Internal working of Emulator:

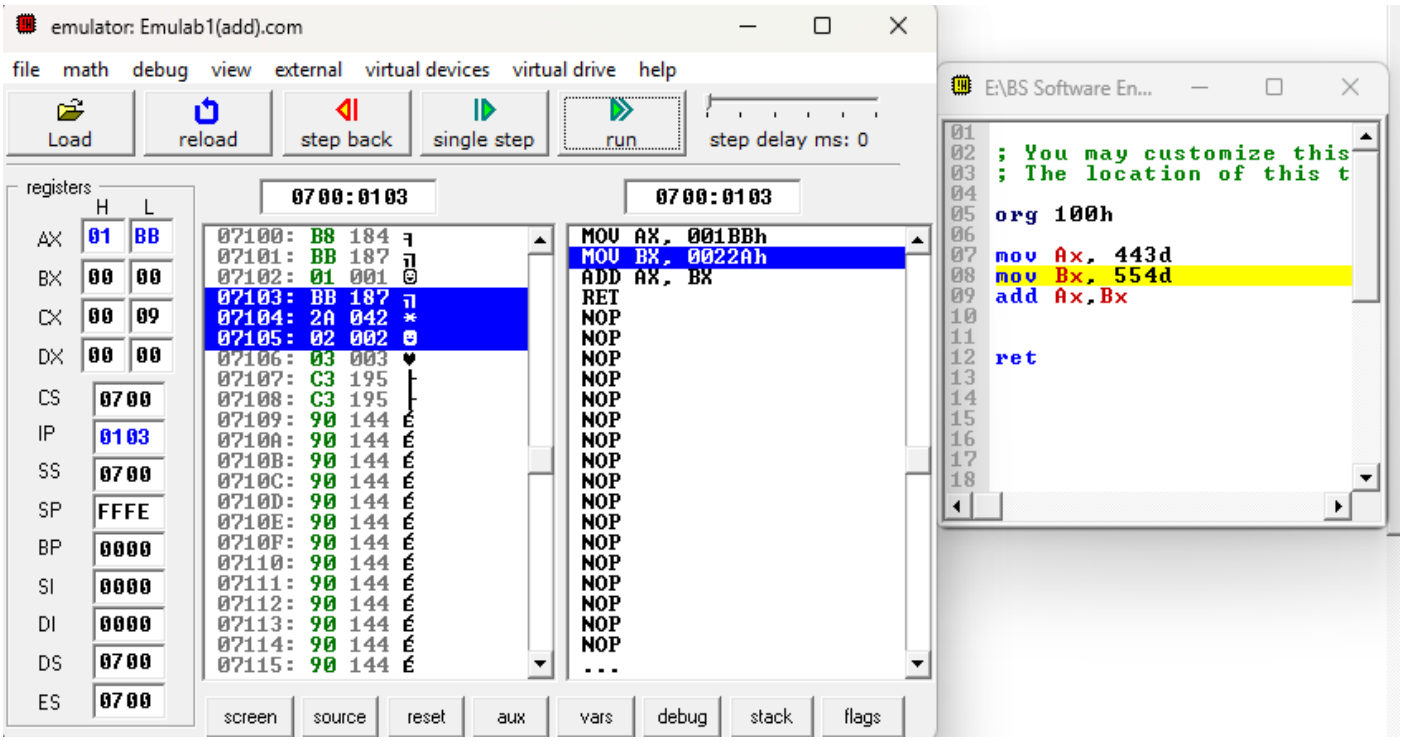
| Register | Decimal | Hexadecimal |
|---|---|---|
| **AX** | 443 | 01BB |
| **BX** | 554 | 022A |

# Compilation:

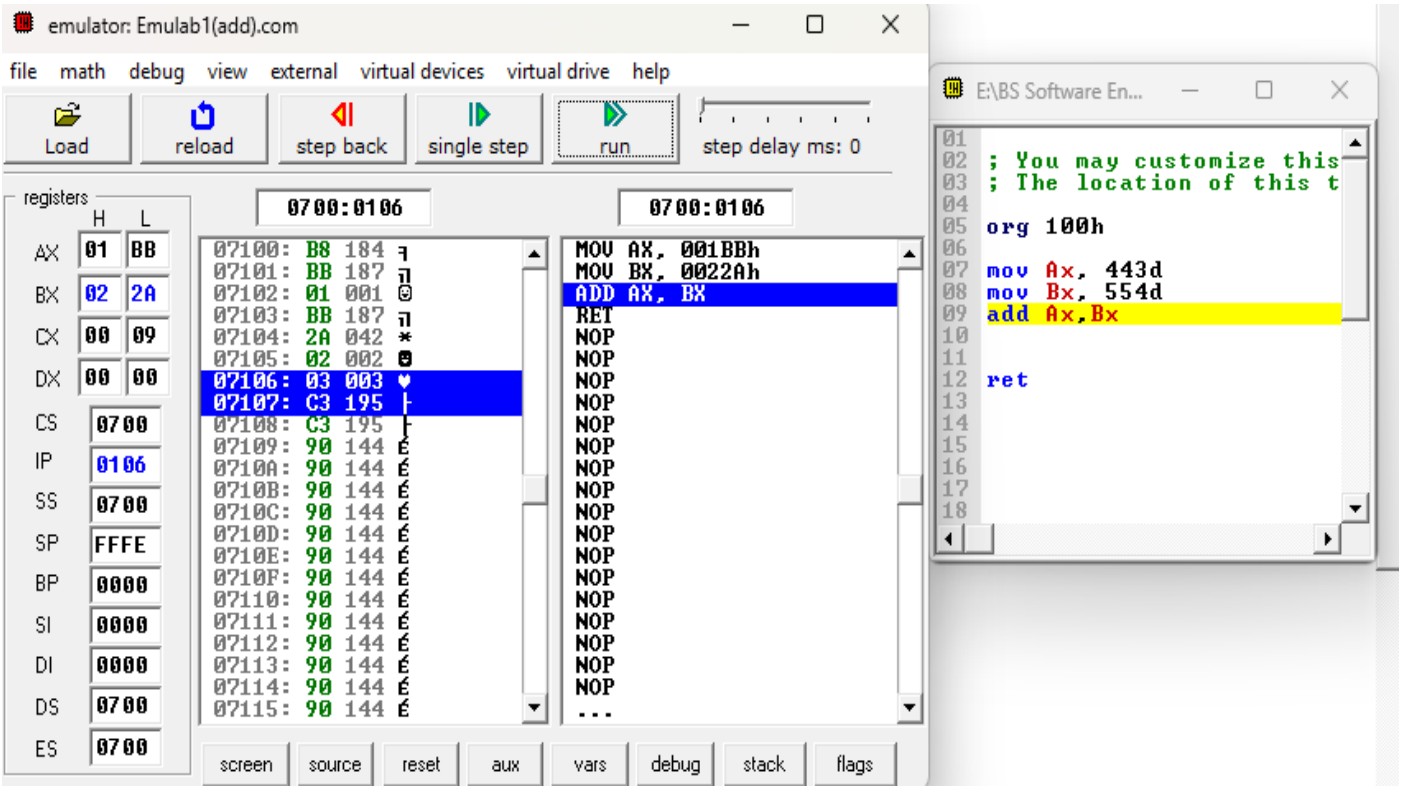
## First Step



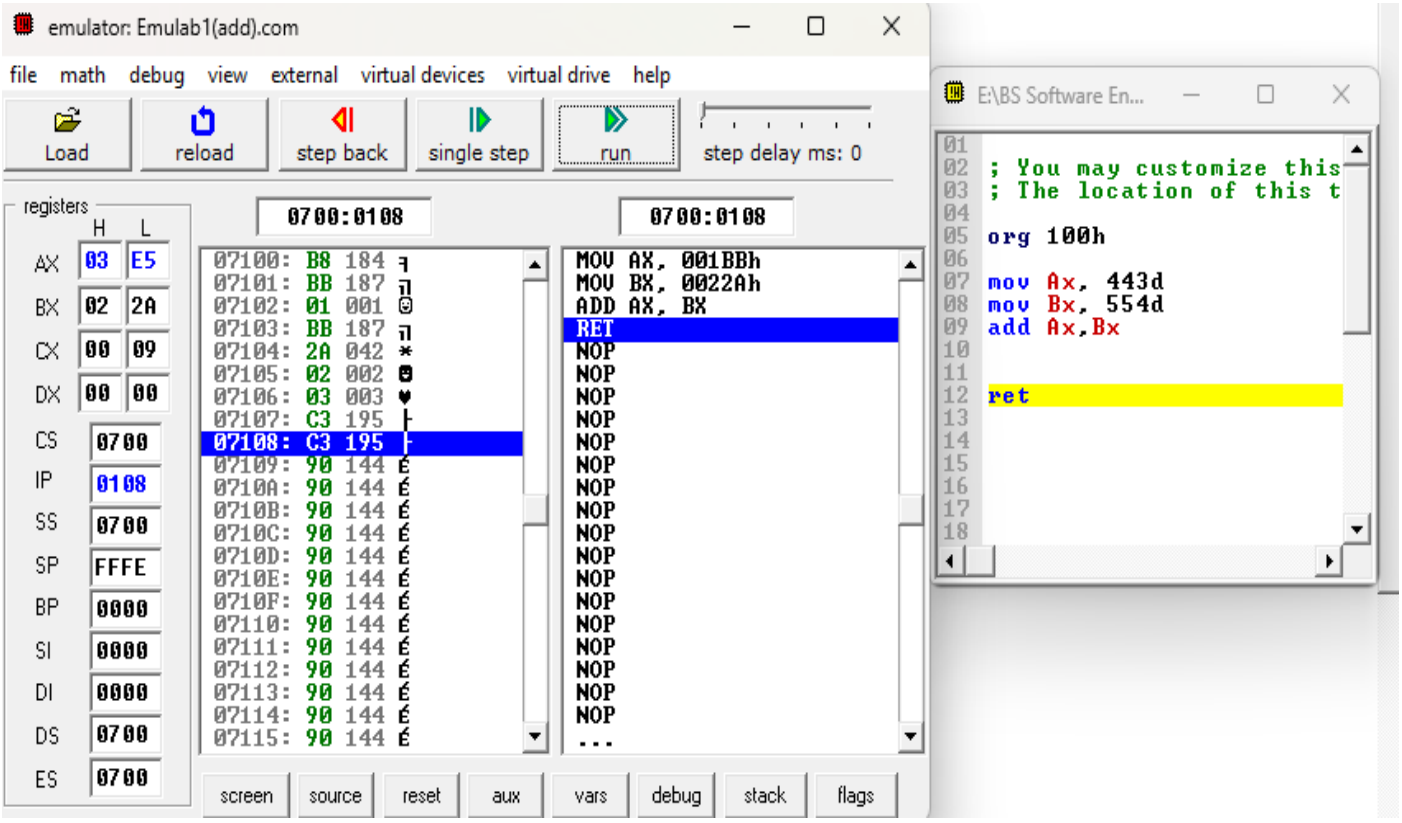

## Value Stored in AX Register



# Value Stored in BX Register




## The Result



**Manually:**

```
  0000000110111011   (AX = 443)
+ 0000001000101010   (BX = 554)
--------------------
  0000001111100101   (AX = 997)

Now converting it in Hex:
Binary 0000001111100101 → Hex 03E5
```

# Subtraction Operation

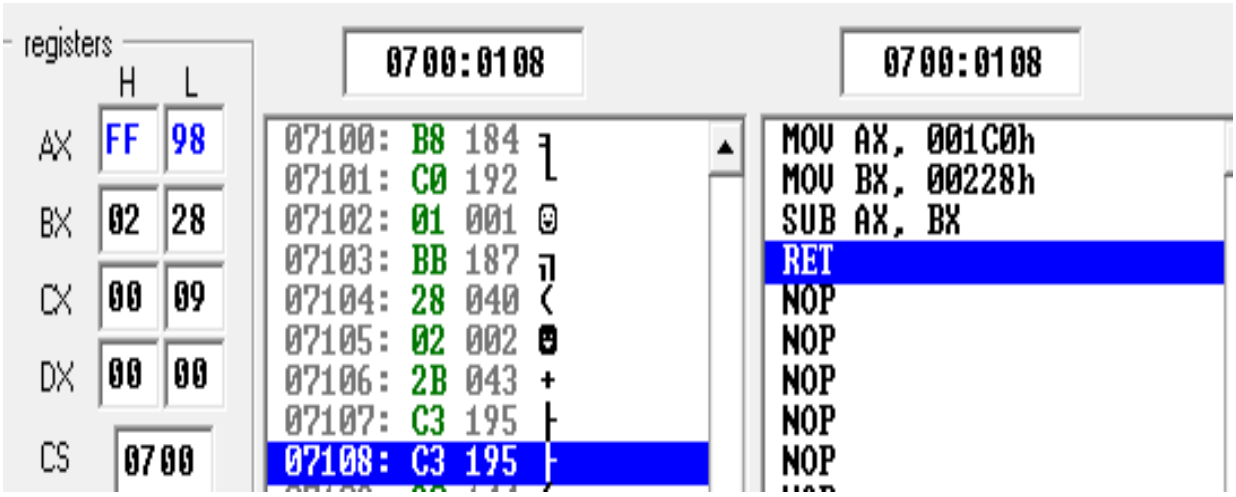
**Original Code:**

```
04
05  org 100h
06
07  mov Ax,448d
08  mov Bx,552d
09  sub Ax,Bx
10
11
12  ret
13
```

## Manually:

```
  0000000111000000   (AX = 448)
-  0000001000101000   (BX = 552)
--------------------
  1111111110011000   (AX = -104 in 2's complement)
Binary 1111111110011000 → Hex FF98

AX = FF98h
```

## The Result



# Multiplication Operation

## Original Code:

E:\BS Software En...

```
01
02  ;  You  may  customize  this
03  ;  The  location  of  this  t
04
05  org  100h
06
07  mov  ax,  9862d      ;  Load
08  mov  bx,  3427d      ;  Load
09  mul  bx
10
11  ret
```

## Manually:

AX = 9862 → 0010011010111110 (binary)
BX = 3427 → 0000110101100011 (binary)
MUL BX → Unsigned multiply: 9862 × 3427 = 33775274
This result is too large for AX alone, so:
AX = lower 16 bits of the result → B3D2h
DX = upper 16 bits → 0203h
So:
DX:AX = 0203B3D2h

## The Result

registers

| | H | L |
|---|---|---|
| AX | B3 | D2 |
| BX | 0D | 63 |
| CX | 00 | 09 |
| DX | 02 | 03 |
| CS | 0700 | |

0700:0108

```
07100:  B8  184  ¶
07101:  86  134  å
07102:  26  038  &
07103:  BB  187  ╗
07104:  63  099  c
07105:  0D  013  CRET
07106:  F7  247  ≈
07107:  E3  227  π
07108:  C3  195  ├
```

0700:0108

```
MOV  AX,  02686h
MOV  BX,  00D63h
MUL  BX
RET
NOP
NOP
NOP
NOP
NOP
```

# Division Operation

## Original Code:

```
org 100h

mov ax, 100        ; AX = 10
mov bx, 20         ; BX = 20
mul bx             ; AX * BX

ret
```

## Manually:

Decimal: $100 \times 20 = 2000$

Hexadecimal: $2000 = 07D0h$

So:

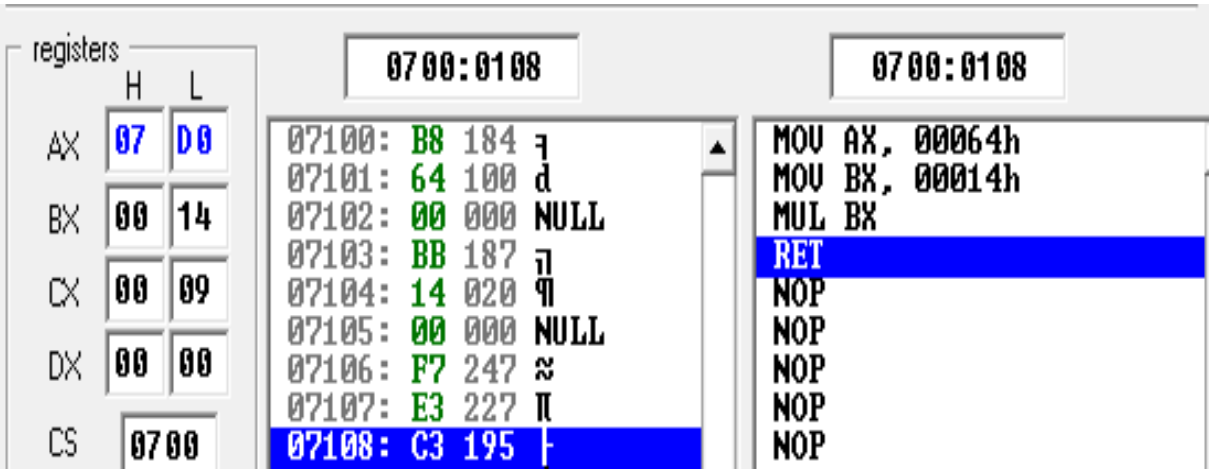
$AX = 07D0h \longrightarrow$ lower 16 bits

$DX = 0000h \longrightarrow$ upper 16 bits

## Emulator Behavior

| Register | Value |
|---|---|
| **Ax** | 07D0 |
| **Dx** | 0000 |

# The Result


registers
H L
AX 07 D0
BX 00 14
CX 00 09
DX 00 00
CS 0700
0700:0108
07100: B8 184
07101: 64 100 d
07102: 00 000 NULL
07103: BB 187
07104: 14 020
07105: 00 000 NULL
07106: F7 247
07107: E3 227
07108: C3 195
0700:0108
MOV AX, 00064h
MOV BX, 00014h
MUL BX
RET
NOP
NOP
NOP
NOP
NOP